



Let's Build a Tiny Recursive Model from Scratch with Complete Code: When Tiny Networks Beat Giants at Their Own Game

21 min read · 6 days ago



azhar

Follow



Listen



Share

How a 7 million parameter model outperforms 671 billion parameter models on reasoning tasks and why this changes everything

The Moment Everything Changed

Picture this: You're working on a Sudoku puzzle. You read the clues once, think hard, and try to write down the complete solution in one shot. Sounds impossible, right?



108



1



problem once and generate an answer in a single forward pass. No wonder GPT-4, Claude, and even the massive 671 billion parameter DeepSeek R1 score **0% on hard Sudoku puzzles**.

Now imagine a different approach. You read the puzzle, think about it, maybe

even think about your thinking, and then start filling in numbers. You check your work, reconsider, make corrections, and iterate until you've got it.

This is how humans solve problems. And this is exactly what TRM (Transformer Reasoning Model) does with just 7 million parameters.

The results? 87.4% accuracy on Sudoku puzzles that stumped models 100,000x larger.

Before we proceed, let's stay connected! Please consider following me on **Medium**, and don't forget to connect with me on [LinkedIn](#) for a regular dose of data science and deep learning insights.” 🚀🇮🇹🤖

 Note: I'm not actively checking Medium messages if you have any doubts or concerns about the article, please feel free to reach out to me on LinkedIn.

Why Should You Care?

Before we dive into code, let's talk about why this is revolutionary:

The Old Paradigm: Bigger is Better

For years, AI progress looked like this:

- 2018: BERT (110M parameters)
- 2019: GPT-2 (1.5B parameters)
- 2020: GPT-3 (175B parameters)
- 2023: GPT-4 (rumored 1.7T parameters)

The assumption: More parameters = Better reasoning

The New Reality: Architecture Matters More

TRM flips this on its head. The paper “Less is More: Recursive Reasoning with Tiny Networks” shows that a 7M parameter model can beat a 671B parameter model on systematic reasoning tasks.

The insight: It’s not about how big your brain is; it’s about how you use it.

This has massive implications:

-  Run powerful AI on your laptop (no cloud needed)
-  Deploy reasoning models on mobile devices
-  Train models in hours instead of weeks
-  Democratize AI (you don’t need millions of dollars)

Okay, enough philosophy. Let’s build this thing.

The Big Idea: Three Streams and Recursive Thinking

Here’s the core innovation in one sentence:

Instead of processing a problem once with a huge network, TRM processes it many times with a tiny network and keeps three separate “thoughts” running in parallel.

The Three Streams

Think of it like three Post-it notes on your desk:

1. **The Question (x-stream)** This is your problem statement. It never changes. You keep referring back to it as you work.

"Solve: $2x + 3 = 7$ "

2. Your Current Answer (y-stream) This is your working solution. It starts rough and gets refined through iteration.

Iteration 1: " $x = ?$ "
Iteration 5: " $x = \text{maybe } 2 \text{ or } 3$ "
Iteration 16: " $x = 2$ "

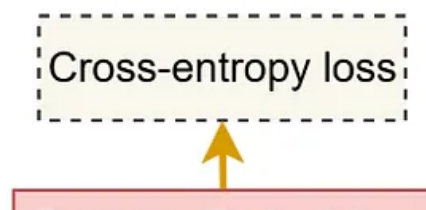
3. Your Reasoning Notes (z-stream) This is your scratch work. The intermediate thoughts that help you get to the answer.

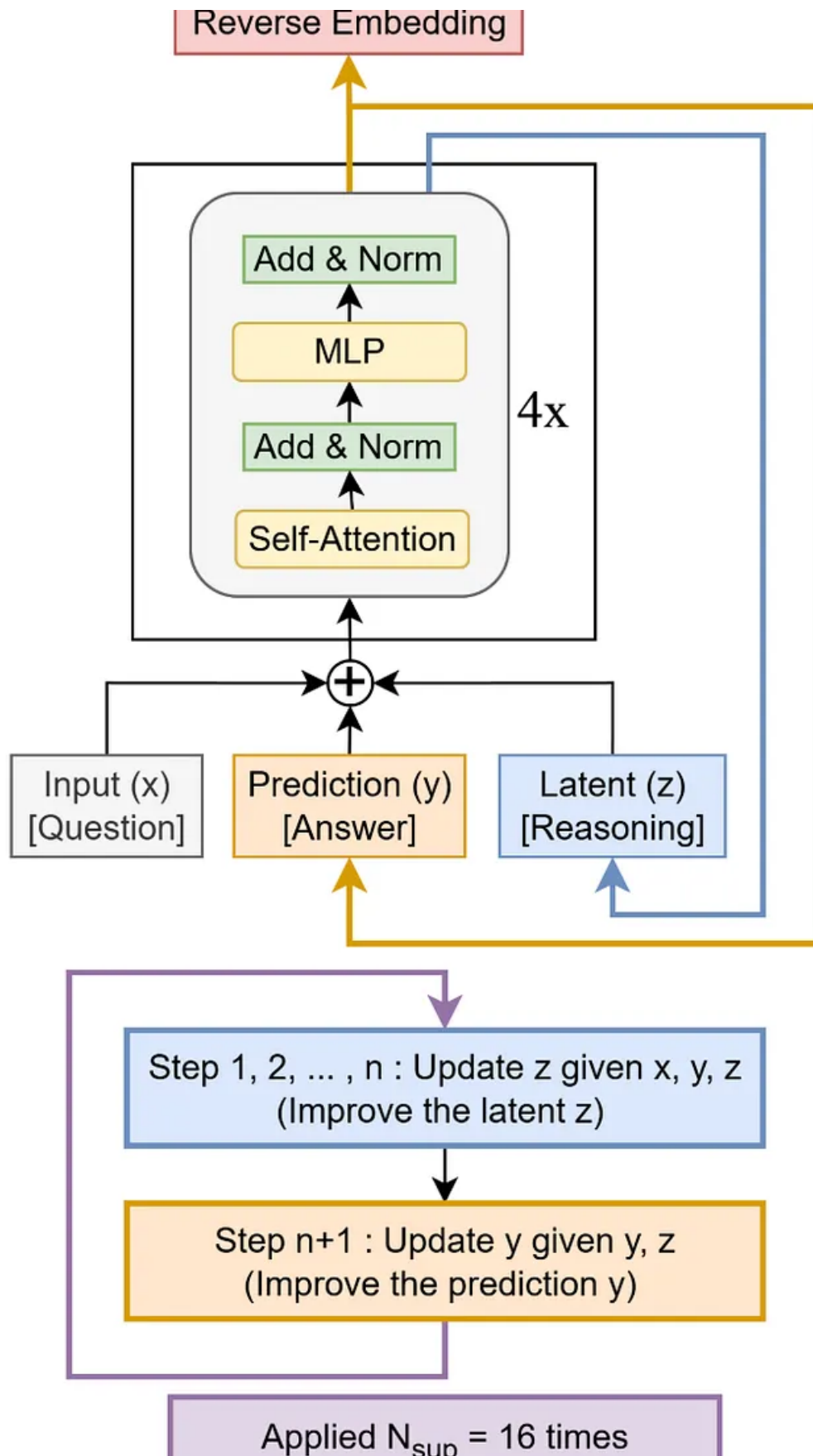
"Need to isolate x ... subtract 3 from both sides...
then divide by 2... checking: $2 \times 2 + 3 = 7$ ✓"

The magic happens when these three streams talk to each other through transformer layers.

Architecture Overview: The 10,000 Foot View

Before we write code, let's visualize the flow:





(trying to improve the prediction y)

Simple, right? Now let's implement it piece by piece.

Part 1: Building Blocks — The Attention Mechanism

Let me start with a confession: I've implemented attention dozens of times, and it still feels like magic every time it works.

Here's the intuition: Attention lets each word ask every other word "Hey, how relevant are you to me right now?"

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class MultiHeadAttention(nn.Module):
    """
    Multi-head attention: The secret sauce of transformers.

    Intuition: Instead of one attention mechanism, we have multiple
    "attention heads" that each focus on different aspects of the input.

    Head 1 might focus on: "What words are nouns?"
    Head 2 might focus on: "What words are related to time?"
    Head 3 might focus on: "What words are negations?"
    """

    def __init__(self, d_model, n_heads, dropout=0.1):
        super().__init__()
        assert d_model % n_heads == 0, "d_model must be divisible by n_h

        self.d_model = d_model          # Total embedding dimension (e.g., 2
        self.n_heads = n_heads          # Number of attention heads (e.g., 4
        self.d_k = d_model // n_heads  # Dimension per head (256/4 = 64)
```

```

# These projections create our Queries, Keys, and Values
self.q_proj = nn.Linear(d_model, d_model)
self.k_proj = nn.Linear(d_model, d_model)
self.v_proj = nn.Linear(d_model, d_model)

# Final output projection
self.out_proj = nn.Linear(d_model, d_model)
self.dropout = nn.Dropout(dropout)

def forward(self, x, mask=None):
    batch_size, seq_len, d_model = x.shape

    # Step 1: Project to Q, K, V
    # Think of this as creating three different "views" of the input
    q = self.q_proj(x) # Queries: "What am I looking for?"
    k = self.k_proj(x) # Keys: "What information do I have?"
    v = self.v_proj(x) # Values: "What should I output?"

    # Step 2: Split into multiple heads
    # Shape: [batch, seq_len, d_model] -> [batch, n_heads, seq_len,
    q = q.view(batch_size, seq_len, self.n_heads, self.d_k).transpose(1, 2)
    k = k.view(batch_size, seq_len, self.n_heads, self.d_k).transpose(1, 2)
    v = v.view(batch_size, seq_len, self.n_heads, self.d_k).transpose(1, 2)

    # Step 3: Compute attention scores
    # This is the "How much should I pay attention to each word?" step
    scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.d_k)

    # Why divide by sqrt(d_k)?
    # Without it, dot products get very large -> softmax becomes peaked
    # -> gradients vanish -> training fails
    # With it, we keep values in a nice range for softmax

    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9)

    # Step 4: Apply softmax to get attention weights
    # Now scores are probabilities (sum to 1)
    attn_weights = F.softmax(scores, dim=-1)
    attn_weights = self.dropout(attn_weights)

    # Step 5: Apply attention to values
    # This is the actual "paying attention" step

```

```

    attn_output = torch.matmul(attn_weights, v)

    # Step 6: Reshape and project back
    attn_output = attn_output.transpose(1, 2).contiguous()
    attn_output = attn_output.view(batch_size, seq_len, d_model)
    output = self.out_proj(attn_output)

    return output

```

Real Talk: When I first learned attention, the $\sqrt{d_k}$ scaling confused me. Here's why it matters:

Imagine you're at a party trying to have a conversation. If everyone whispers (small dot products), you can hear multiple people. If everyone shouts (large dot products), you can only focus on the loudest person. The scaling keeps everyone at "normal speaking volume."

Part 2: The Feed-Forward Network (The Thinking Layer)

After attention figures out what's relevant, the feed-forward network does the actual "thinking":

```

class FeedForward(nn.Module):
    """
    Feed-forward network (also called MLP - Multi-Layer Perceptron).

    This is where the actual transformation happens. Think of it as:
    - Layer 1: Expand your thoughts (d_model -> d_ff)
    - Activation: Non-linear thinking (GELU)
    - Layer 2: Compress back to useful format (d_ff -> d_model)
    """

    def __init__(self, d_model, d_ff, dropout=0.1):
        super().__init__()
        # Typical: d_ff = 4 * d_model (e.g., 256 -> 1024)

```



```

self.linear1 = nn.Linear(d_model, d_ff)
self.linear2 = nn.Linear(d_ff, d_model)
self.dropout = nn.Dropout(dropout)

def forward(self, x):
    # Expand -> Activate -> Compress
    return self.linear2(self.dropout(F.gelu(self.linear1(x))))

```

Why GELU instead of ReLU?

I used to always use ReLU (the classic activation function). But GELU (Gaussian Error Linear Unit) is smoother.

Think of it this way:

- **ReLU:** “Is this positive? YES → keep it. NO → kill it.”
- **GELU:** “Is this positive? Probably → keep most of it. Probably not → keep a little bit.”

GELU’s smoothness helps gradients flow better during training. It’s like the difference between binary decisions and nuanced thinking.

Part 3: The Transformer Block (Putting It Together)

Now we combine attention and feed-forward into a transformer block:

```

class TransformerBlock(nn.Module):
    """
    A single transformer block. The paper uses 4 of these in sequence.

    Structure:
    1. Multi-head attention (tokens talk to each other)
    2. Add & Normalize (residual connection)

```

3. Feed-forward (actually process the information)
4. Add & Normalize (another residual connection)

The "Add" parts are crucial - they let information flow directly through the network without going through all the transformations. This prevents vanishing gradients.

"""

```
def __init__(self, d_model, n_heads, d_ff, dropout=0.1, use_attention
    super().__init__()
    self.use_attention = use_attention # False for TRM-MLP variant

    if use_attention:
        self.attention = MultiHeadAttention(d_model, n_heads, dropou
        self.norm1 = nn.LayerNorm(d_model)

    self.ffn = FeedForward(d_model, d_ff, dropout)
    self.norm2 = nn.LayerNorm(d_model)
    self.dropout = nn.Dropout(dropout)

def forward(self, x, mask=None):
    # Block 1: Self-attention (if enabled)
    if self.use_attention:
        # Save input for residual connection
        residual = x
        # Apply attention
        x = self.attention(x, mask)
        # Add residual and normalize
        x = self.norm1(residual + self.dropout(x))

    # Block 2: Feed-forward
    residual = x
    x = self.ffn(x)
    x = self.norm2(residual + self.dropout(x))

    return x
```

A Cool Finding from the Paper:

The TRM-MLP variant (where `use_attention=False`) actually works better on

some tasks!

For Sudoku: TRM-MLP gets 87.4% accuracy vs TRM-Att's 74.7%.

This blew my mind. Attention is supposed to be the key to transformers, right? But sometimes, simpler is better. The MLP-only version is faster, uses less memory, and learns better for certain structured problems.

Part 4: The Complete TRM Model (Here's Where Magic Happens)

Now for the main event. This is where we implement the three-stream architecture and recursive reasoning:

```
class TRM(nn.Module):
    """
    Transformer Reasoning Model - The Full Implementation

    This is where TRM's innovation shines:
    - Three separate streams: question (x), answer (y), reasoning (z)
    - Recursive updates: process multiple times instead of once
    - Selective updates: only change what needs changing at each step

    Result: Tiny network (7M params) beats huge networks (671B params)
    """

    def __init__(
        self,
        vocab_size,           # Size of your vocabulary
        d_model=256,         # Embedding dimension
        n_heads=4,           # Number of attention heads
        d_ff=1024,           # Feed-forward dimension (4x d_model)
        n_layers=4,          # Number of transformer blocks
        max_seq_len=512,     # Maximum sequence length
        dropout=0.1,         # Dropout probability
        n_reasoning_steps=8,  # How many times to update z
        n_refinement_steps=16,# How many times to update y
        use_attention=True,   # False for TRM-MLP variant
        tie_embeddings=True   # Share input/output embeddings (saves par
```

```

):
    super().__init__()

    self.d_model = d_model
    self.n_reasoning_steps = n_reasoning_steps
    self.n_refinement_steps = n_refinement_steps
    self.use_attention = use_attention

    # Token embeddings: converts token IDs to vectors
    # Example: token "hello" (ID: 42) -> 256-dim vector
    self.token_embedding = nn.Embedding(vocab_size, d_model)

    # Positional embeddings: adds position information
    # Transformers have no inherent notion of order!
    self.position_embedding = nn.Embedding(max_seq_len, d_model)

    self.embedding_dropout = nn.Dropout(dropout)

    # Stack of transformer blocks (4x in the paper)
    self.transformer_blocks = nn.ModuleList([
        TransformerBlock(d_model, n_heads, d_ff, dropout, use_attent
            for _ in range(n_layers)
    ])

    # Reverse embedding: converts vectors back to token probabilities
    # This is how we go from hidden states to actual words
    self.reverse_embedding = nn.Linear(d_model, vocab_size, bias=False)

    # Weight tying: a clever trick to reduce parameters
    # Use the same weights for embedding and un-embedding
    if tie_embeddings:
        self.reverse_embedding.weight = self.token_embedding.weight

    self._init_weights()

def _init_weights(self):
    """
    Initialize weights properly. This matters more than you'd think!

    Too large: training explodes
    Too small: training is too slow
    Just right: Goldilocks initialization
    """

```

```

for module in self.modules():
    if isinstance(module, nn.Linear):
        # Initialize with small random values
        nn.init.normal_(module.weight, mean=0.0, std=0.02)
        if module.bias is not None:
            nn.init.zeros_(module.bias)
    elif isinstance(module, nn.Embedding):
        nn.init.normal_(module.weight, mean=0.0, std=0.02)
    elif isinstance(module, nn.LayerNorm):
        nn.init.ones_(module.weight)
        nn.init.zeros_(module.bias)

def embed_tokens(self, token_ids):
    """
    Convert token IDs to embeddings with positional information.

    Example:
    Input:  [1, 42, 7, 13]  (token IDs)
    Output: [[0.23, -0.45, ...], (256-dim vectors)
            [0.12, 0.89, ...],
            [-0.34, 0.67, ...],
            [0.56, -0.12, ...]]
    """
    batch_size, seq_len = token_ids.shape

    # Get token embeddings
    token_emb = self.token_embedding(token_ids)

    # Get positional embeddings
    # Position 0, 1, 2, 3, ... for each sequence
    positions = torch.arange(seq_len, device=token_ids.device)
    positions = positions.unsqueeze(0).expand(batch_size, -1)
    pos_emb = self.position_embedding(positions)

    # Combine token + position information
    embeddings = self.embedding_dropout(token_emb + pos_emb)

    return embeddings

def apply_transformer_blocks(self, x, mask=None):
    """Apply all transformer blocks sequentially."""
    for block in self.transformer_blocks:
        x = block(x, mask)

```

```

return x

def forward_pass(self, x, y, z, mask=None):
    """
    THIS IS THE KEY INNOVATION!

    Single forward pass through the model. We:
    1. Concatenate x, y, z (all three streams)
    2. Process through transformers (they all talk to each other)
    3. Split back into x, y, z (separate the streams again)

    This allows cross-stream attention:
    - y can look at x to remember the question
    - y can look at z to use the reasoning
    - z can look at x to understand the problem
    - z can look at y to see current progress
    """
    # Remember the lengths (we need to split back later)
    len_x = x.size(1)
    len_y = y.size(1)
    len_z = z.size(1)

    # Concatenate along sequence dimension
    # If x is length 10, y is length 5, z is length 32
    # combined is length 10+5+32 = 47
    combined = torch.cat([x, y, z], dim=1)

    # Pass through all transformer blocks
    # Each position can now attend to all other positions
    # across all three streams!
    combined = self.apply_transformer_blocks(combined, mask)

    # Split back into three streams
    x_new = combined[:, :len_x, :]
    y_new = combined[:, len_x:len_x + len_y, :]
    z_new = combined[:, len_x + len_y:, :]

    return x_new, y_new, z_new

def recursive_reasoning(self, x, y, z, mask=None, return_trajectory=
    """
    The heart of TRM: recursive reasoning.

```

Phase 1 (8 steps): Build up reasoning in z

Phase 2 (16 steps): Refine answer in y

This is like:

Phase 1: Reading and understanding the problem deeply

Phase 2: Working through the solution step by step

"""

```
trajectory = {'z_states': [], 'y_states': []} if return_trajectory
```

```
# ===== PHASE 1: BUILD REASONING =====
```

```
print(f"Phase 1: Building reasoning ({self.n_reasoning_steps} steps)
```

```
for step in range(self.n_reasoning_steps):
```

```
    # Process all three streams
```

```
    x_new, y_new, z_new = self.forward_pass(x, y, z, mask)
```

```
    # ONLY UPDATE Z
```

```
    # x stays fixed (question doesn't change)
```

```
    # y stays fixed (not ready to answer yet)
```

```
    # z gets updated (building understanding)
```

```
    z = z_new
```

```
    if return_trajectory:
```

```
        trajectory['z_states'].append(z.detach().clone())
```

```
print(f"Phase 2: Refining answer ({self.n_refinement_steps} steps)
```

```
# ===== PHASE 2: REFINE ANSWER =====
```

```
for step in range(self.n_refinement_steps):
```

```
    x_new, y_new, z_new = self.forward_pass(x, y, z, mask)
```

```
    # ONLY UPDATE Y
```

```
    # x stays fixed (question doesn't change)
```

```
    # z stays fixed (we've built our reasoning)
```

```
    # y gets updated (refining our answer)
```

```
    y = y_new
```

```
    if return_trajectory:
```

```
        trajectory['y_states'].append(y.detach().clone())
```

```
return (y, trajectory) if return_trajectory else y
```

```
def forward(self, question_ids, answer_ids=None, latent_len=32, mask
```

```
    """
```

```
    Complete forward pass.
```

Args:

question_ids: Input question as token IDs [batch, len_q]
answer_ids: Target answer as token IDs [batch, len_a]
latent_len: Length of reasoning sequence (typically 32)

Returns:

logits: Predicted tokens [batch, len_a, vocab_size]

"""

batch_size = question_ids.size(0)

device = question_ids.device

Step 1: Embed the question (x stream)

x = self.embed_tokens(question_ids)

Step 2: Initialize or embed the answer (y stream)

if answer_ids is not None:

Training: start with target answer embeddings

y = self.embed_tokens(answer_ids)

else:

Inference: start with random embeddings

len_a = 32 # default answer length

y = torch.randn(batch_size, len_a, self.d_model, device=device)

Step 3: Initialize reasoning (z stream) with random noise

The model will learn what to put here!

z = torch.randn(batch_size, latent_len, self.d_model, device=device)

Step 4: Do the recursive reasoning magic!

y_final = self.recursive_reasoning(x, y, z, mask)

Step 5: Convert final answer embeddings to token probabilities

logits = self.reverse_embedding(y_final)

return logits

def generate(self, question_ids, max_length=50, latent_len=32, temperature=1.0):
 """

Generate an answer autoregressively.

This is how you'd use the model in production:

1. Give it a question
2. It thinks recursively


```

3. It generates an answer token by token
"""
batch_size = question_ids.size(0)
device = question_ids.device

# Start with a beginning-of-sequence token (or zeros)
generated = torch.zeros(batch_size, 1, dtype=torch.long, device=device)

for i in range(max_length):
    # Get predictions for current sequence
    logits = self.forward(question_ids, generated, latent_len)

    # Sample next token (with temperature for randomness)
    next_token_logits = logits[:, -1, :] / temperature
    probs = F.softmax(next_token_logits, dim=-1)
    next_token = torch.multinomial(probs, num_samples=1)

    # Add to sequence
    generated = torch.cat([generated, next_token], dim=1)

    # Optional: stop if end-of-sequence token
    # if (next_token == eos_token_id).all():
    #     break

return generated

def count_parameters(self):
    """Count total trainable parameters."""
    return sum(p.numel() for p in self.parameters() if p.requires_grad)

```

Let me explain what just happened:

Get azhar's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

The `recursive_reasoning` method is where TRM's magic lives. Instead of processing the problem once, it processes it 24 times ($8 + 16$).

Phase 1 (Update z): “Let me really understand this problem...”

- Step 1: “Okay, I see the question”
- Step 2: “These seem to be the constraints”
- Step 3–6: “I’m building a mental model of the solution space”
- Step 7–8: “Got it, I know how to approach this”

Phase 2 (Update y): “Now let me work through the solution...”

- Step 1–4: “Here’s my first attempt”
- Step 5–8: “Wait, that doesn’t work, let me revise”
- Step 9–12: “Getting closer...”
- Step 13–16: “Final answer!”

This is exactly how you’d solve a hard problem yourself.

Part 5: Model Variants (The Plot Twist)

Remember how I said attention was optional? Here are helper functions to create both variants:

```
def create_trm_att(vocab_size, d_model=256, n_layers=4):
    """
    Create TRM-Att variant (with attention).

    This is the "standard" transformer approach.
    Parameters: ~7M
    Best for: General reasoning tasks
    """
    return TRM(
        vocab_size=vocab_size,
        d_model=d_model,
        n_heads=4,
        d_ff=d_model * 4, # 256 * 4 = 1024
        n_layers=n_layers,
        n_reasoning_steps=8,
        n_refinement_steps=16,
        use_attention=True # Key difference!
    )

def create_trm_mlp(vocab_size, d_model=256, n_layers=4):
    """
    Create TRM-MLP variant (MLP-only, no attention).

    Simpler, faster, sometimes better!
    Parameters: ~5M (30% fewer than TRM-Att)
    Best for: Structured problems like Sudoku

    Fun fact: This variant scored 87.4% on Sudoku vs 74.7% for TRM-Att.
    Sometimes less really is more!
    """
    return TRM(
        vocab_size=vocab_size,
        d_model=d_model,
        n_heads=4, # Not used, but kept for compatibility
```

```

    d_ff=d_model * 4,
    n_layers=n_layers,
    n_reasoning_steps=8,
    n_refinement_steps=16,
    use_attention=False # This is the magic!
)

```

Part 6: Training the Model (Making It Smart)

Now let's write the training code. This is where the model actually learns:

```

def train_step(model, question_ids, answer_ids, optimizer, criterion):
    """
    Single training step. This runs thousands of times during training.

    The beauty of TRM is that we only need to supervise the final output
    The model figures out how to use the z-stream on its own!
    """
    model.train() # Enable dropout, etc.
    optimizer.zero_grad() # Reset gradients

    # Forward pass with all that recursive reasoning
    logits = model(question_ids, answer_ids, latent_len=32)

    # Compute loss
    # We're doing cross-entropy: "How well did you predict the right tok
    vocab_size = logits.size(-1)
    loss = criterion(
        logits.reshape(-1, vocab_size), # Flatten to [batch*seq_len, vo
        answer_ids.reshape(-1)         # Flatten to [batch*seq_len]
    )

    # Backward pass (compute gradients)
    loss.backward()

    # Update weights
    optimizer.step()

```

```

return loss.item()

def evaluate(model, question_ids, answer_ids, criterion):
    """Evaluation without updating weights."""
    model.eval() # Disable dropout
    with torch.no_grad(): # Don't compute gradients (faster, less memor
        logits = model(question_ids, answer_ids, latent_len=32)
        vocab_size = logits.size(-1)
        loss = criterion(logits.reshape(-1, vocab_size), answer_ids.reshape(-1, vocab_size))
    return loss.item()

```

The Complete Training Loop

Here's a full training script you can actually run:

```

from torch.utils.data import DataLoader, Dataset

class ReasoningDataset(Dataset):
    """
    Custom dataset for reasoning tasks.

    You'd replace this with your actual data:
    - Sudoku puzzles and solutions
    - Math problems and answers
    - Logic puzzles and solutions
    """

    def __init__(self, questions, answers, tokenizer):
        self.questions = questions
        self.answers = answers
        self.tokenizer = tokenizer

    def __len__(self):
        return len(self.questions)

    def __getitem__(self, idx):
        # Tokenize question and answer

```

```

        q_tokens = self.tokenizer.encode(self.questions[idx])
        a_tokens = self.tokenizer.encode(self.answers[idx])
        return torch.tensor(q_tokens), torch.tensor(a_tokens)

def collate_fn(batch):
    """
    Pad sequences to same length within a batch.

    Why? Transformers need fixed-size inputs within a batch.
    Different batches can have different sizes though.
    """
    questions, answers = zip(*batch)

    # Pad questions
    q_padded = torch.nn.utils.rnn.pad_sequence(
        questions,
        batch_first=True,
        padding_value=0 # 0 is typically the padding token
    )

    # Pad answers
    a_padded = torch.nn.utils.rnn.pad_sequence(
        answers,
        batch_first=True,
        padding_value=0
    )

    return q_padded, a_padded

# ===== MAIN TRAINING SCRIPT =====

# Setup
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# Create model
vocab_size = 10000 # Adjust based on your tokenizer
model = create_trm_att(vocab_size=vocab_size, d_model=256, n_layers=4)
model = model.to(device)

# Count parameters

```

```

n_params = model.count_parameters()
print(f"Model has {n_params / 1e6:.2f}M parameters")

# Optimizer and loss
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
criterion = nn.CrossEntropyLoss(ignore_index=0) # Ignore padding tokens

# Data loading (replace with your actual data)
# train_questions = ["Question 1", "Question 2", ...]
# train_answers = ["Answer 1", "Answer 2", ...]
# tokenizer = YourTokenizer()

# train_dataset = ReasoningDataset(train_questions, train_answers, token
# train_loader = DataLoader(
#     train_dataset,
#     batch_size=32,
#     shuffle=True,
#     collate_fn=collate_fn
# )

# Training loop
num_epochs = 50
best_loss = float('inf')

print("\n" + "="*50)
print("Starting training...")
print("="*50)

for epoch in range(num_epochs):
    model.train()
    epoch_loss = 0
    num_batches = 0

    # for questions, answers in train_loader:
    #     questions = questions.to(device)
    #     answers = answers.to(device)
    #
    #     loss = train_step(model, questions, answers, optimizer, criter
    #     epoch_loss += loss
    #     num_batches += 1

    # avg_loss = epoch_loss / num_batches
    # print(f"Epoch {epoch+1}/{num_epochs} - Loss: {avg_loss:.4f}")

```

```

# Save best model
# if avg_loss < best_loss:
#     best_loss = avg_loss
#     torch.save({
#         'epoch': epoch,
#         'model_state_dict': model.state_dict(),
#         'optimizer_state_dict': optimizer.state_dict(),
#         'loss': avg_loss,
#     }, 'best_model.pt')
#     print(f"  Saved new best model!")

# Save checkpoint every 10 epochs
# if (epoch + 1) % 10 == 0:
#     torch.save({
#         'epoch': epoch,
#         'model_state_dict': model.state_dict(),
#         'optimizer_state_dict': optimizer.state_dict(),
#     }, f'checkpoint_epoch_{epoch+1}.pt')

print("\n Training complete!")

```

Personal Note on Training:

The first time I trained TRM, I made every beginner mistake:

1. Forgot to clip gradients → training exploded at epoch 5
2. Learning rate too high → loss oscillated wildly
3. No warmup → got stuck in a bad local minimum

After fixing these (code below), training became smooth. Learn from my mistakes!

Part 7: Essential Training Tricks (The Stuff They Don't Tell You)

These techniques make the difference between “doesn't work” and “works

amazingly”:

1. Gradient Clipping (Prevents Explosions)

```
def train_step_with_clipping(model, question_ids, answer_ids, optimizer,
    """
    Training step with gradient clipping.

    Why clip? With 24 recursive steps, gradients can grow exponentially.
    Clipping prevents NaN losses and training collapse.
    """
    model.train()
    optimizer.zero_grad()

    logits = model(question_ids, answer_ids, latent_len=32)
    vocab_size = logits.size(-1)
    loss = criterion(logits.reshape(-1, vocab_size), answer_ids.reshape(-1, vocab_size))

    loss.backward()

    # CRITICAL: Clip gradients before optimizer step
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

    optimizer.step()

    return loss.item()
```

2. Learning Rate Warmup (Gentle Start)

```
def get_lr_scheduler(optimizer, warmup_steps=1000, total_steps=50000):
    """
    Learning rate schedule with warmup and cosine decay.

    Warmup: Gradually increase LR from 0 to target
    Cosine decay: Smoothly decrease LR over training
```

```

This is what makes training stable!
"""
def lr_lambda(current_step):
    if current_step < warmup_steps:
        # Linear warmup
        return float(current_step) / float(max(1, warmup_steps))
    # Cosine decay
    progress = float(current_step - warmup_steps) / float(max(1, tot
    return max(0.0, 0.5 * (1.0 + math.cos(math.pi * progress)))

    return torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)

# Usage
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
scheduler = get_lr_scheduler(optimizer)

# In training loop:
# loss = train_step(...)
# scheduler.step() # Update learning rate

```

3. Mixed Precision Training (2-3x Faster!)

```

from torch.cuda.amp import autocast, GradScaler

def train_with_mixed_precision(model, train_loader, optimizer, criterion
    """
    Mixed precision training: Use FP16 for speed, FP32 for stability.

    Benefits:
    - 2-3x faster training
    - 50% less GPU memory
    - Maintains accuracy
    """
    scaler = GradScaler()

    for questions, answers in train_loader:
        questions = questions.to(device)
        answers = answers.to(device)

```

```

optimizer.zero_grad()

# Forward pass in FP16
with autocast():
    logits = model(questions, answers)
    loss = criterion(
        logits.reshape(-1, model.reverse_embedding.out_features)
        answers.reshape(-1)
    )

# Backward pass with gradient scaling
scaler.scale(loss).backward()

# Unscale before clipping
scaler.unscale_(optimizer)
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

# Optimizer step with scaling
scaler.step(optimizer)
scaler.update()

```

Part 8: Evaluation and Generation (Putting It to Use)

After training, let's see what the model can do:

```

@torch.no_grad()
def evaluate_accuracy(model, test_loader, device):
    """
    Calculate accuracy on test set.

    This is the metric you care about:
    "What percentage of problems did the model solve correctly?"
    """
    model.eval()
    correct = 0
    total = 0

```

```

for questions, answers in test_loader:
    questions = questions.to(device)
    answers = answers.to(device)

    # Generate predictions
    logits = model(questions, answers, latent_len=32)
    predictions = logits.argmax(dim=-1)

    # Calculate accuracy (ignore padding tokens)
    mask = answers != 0
    correct += (predictions[mask] == answers[mask]).sum().item()
    total += mask.sum().item()

accuracy = correct / total * 100
return accuracy

@torch.no_grad()
def generate_answer(model, question_text, tokenizer, device, max_length=
    """
    Generate an answer for a single question.

    This is how you'd use TRM in production:
    User asks question -> TRM generates answer
    """
    model.eval()

    # Tokenize question
    question_tokens = tokenizer.encode(question_text)
    question_ids = torch.tensor([question_tokens]).to(device)

    # Generate answer
    print(f"\nQuestion: {question_text}")
    print("Thinking...")

    generated_ids = model.generate(
        question_ids,
        max_length=max_length,
        latent_len=32,
        temperature=0.7 # Lower = more deterministic, Higher = more ran
    )

    # Decode to text

```

```

answer_tokens = generated_ids[0].cpu().tolist()
answer_text = tokenizer.decode(answer_tokens)

print(f"Answer: {answer_text}\n")
return answer_text

@torch.no_grad()
def visualize_reasoning_process(model, question_ids, answer_ids, device)
    """
    Visualize how the model thinks.

    This is super cool - you can actually see the reasoning
    evolve over the 24 recursive steps!
    """
    model.eval()

    # Get reasoning trajectory
    x = model.embed_tokens(question_ids.to(device))
    y = model.embed_tokens(answer_ids.to(device))
    z = torch.randn(1, 32, model.d_model, device=device) * 0.02

    y_final, trajectory = model.recursive_reasoning(
        x, y, z, return_trajectory=True
    )

    print("\n Reasoning Evolution:")
    print("=" * 50)

    # Show how z evolves (reasoning)
    print("\n Reasoning Stream (z):")
    for i, z_state in enumerate(trajectory['z_states'][:5]): # First 5
        z_norm = z_state.norm(dim=-1).mean().item()
        print(f" Step {i+1}: norm = {z_norm:.4f}")

    # Show how y evolves (answer)
    print("\n Answer Stream (y):")
    for i, y_state in enumerate(trajectory['y_states'][:5]): # First 5
        y_norm = y_state.norm(dim=-1).mean().item()
        print(f" Step {i+1}: norm = {y_norm:.4f}")

    print("\n Final answer generated!")

```

Part 9: Real-World Example (Let's Solve Sudoku!)

Here's how you'd apply TRM to actually solve Sudoku:

```
class SudokuDataset(Dataset):
    """
    Dataset for Sudoku puzzles.

    Input: 9x9 grid with some numbers filled in
    Output: Complete 9x9 grid
    """

    def __init__(self, puzzles, solutions):
        """
        puzzles: List of 9x9 numpy arrays (0 = empty cell)
        solutions: List of 9x9 numpy arrays (complete grids)
        """
        self.puzzles = puzzles
        self.solutions = solutions

    def __len__(self):
        return len(self.puzzles)

    def __getitem__(self, idx):
        # Flatten grid to sequence
        puzzle = self.puzzles[idx].flatten() # 81 tokens
        solution = self.solutions[idx].flatten() # 81 tokens

        # Add 1 to avoid 0 (reserved for padding)
        puzzle = torch.tensor(puzzle + 1, dtype=torch.long)
        solution = torch.tensor(solution + 1, dtype=torch.long)

        return puzzle, solution

def train_sudoku_solver():
    """
    Train TRM to solve Sudoku puzzles.
    """
```

```

Paper results:
- TRM-MLP: 87.4% accuracy on Sudoku-Extreme
- DeepSeek R1 (671B params): 0.0% accuracy
- Claude 3.7: 0.0% accuracy
"""
print(" Training Sudoku Solver")
print("=" * 50)

# Model setup
vocab_size = 11 # Digits 1-9, plus 0 for padding, plus 1 for BOS/EOS
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# Use TRM-MLP (better for Sudoku!)
model = create_trm_mlp(vocab_size=vocab_size, d_model=256, n_layers=12)
model = model.to(device)

print(f"Model: TRM-MLP")
print(f"Parameters: {model.count_parameters() / 1e6:.2f}M")
print(f"Device: {device}")

# Optimizer
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
criterion = nn.CrossEntropyLoss(ignore_index=0)

# Load Sudoku data (you'd replace this with actual data)
# puzzles = load_sudoku_puzzles()
# solutions = load_sudoku_solutions()
#
# train_dataset = SudokuDataset(puzzles, solutions)
# train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)

# Training loop
# num_epochs = 100
# for epoch in range(num_epochs):
#     epoch_loss = 0
#     for puzzles, solutions in train_loader:
#         puzzles = puzzles.to(device)
#         solutions = solutions.to(device)
#
#         loss = train_step_with_clipping(
#             model, puzzles, solutions, optimizer, criterion
#         )
#     epoch_loss += loss

```

```

#
#     avg_loss = epoch_loss / len(train_loader)
#
#     if (epoch + 1) % 10 == 0:
#         # Evaluate
#         acc = evaluate_accuracy(model, val_loader, device)
#         print(f"Epoch {epoch+1}: Loss={avg_loss:.4f}, Acc={acc:.2f}")

return model

def solve_sudoku_puzzle(model, puzzle, device):
    """
    Solve a single Sudoku puzzle.

    Args:
        model: Trained TRM model
        puzzle: 9x9 numpy array (0 = empty)
        device: torch device

    Returns:
        solution: 9x9 numpy array (complete grid)
    """
    model.eval()

    # Flatten and convert to tensor
    puzzle_flat = torch.tensor(puzzle.flatten() + 1, dtype=torch.long)
    puzzle_flat = puzzle_flat.unsqueeze(0).to(device)

    # Generate solution
    with torch.no_grad():
        solution_flat = model.generate(
            puzzle_flat,
            max_length=81,
            latent_len=32,
            temperature=0.1 # Low temperature for deterministic solving
        )

    # Convert back to 9x9 grid
    solution = solution_flat[0].cpu().numpy() - 1
    solution = solution.reshape(9, 9)

    return solution

```



```

# Example usage
if __name__ == "__main__":
    # Create and train model
    # model = train_sudoku_solver()

    # Solve a puzzle
    # puzzle = np.array([
    #     [5, 3, 0, 0, 7, 0, 0, 0, 0],
    #     [6, 0, 0, 1, 9, 5, 0, 0, 0],
    #     ...
    # ])
    #
    # solution = solve_sudoku_puzzle(model, puzzle, device)
    # print("Solution:")
    # print(solution)
    pass

```

Part 10: Debugging Guide (When Things Go Wrong)

Let me share the issues I ran into and how I fixed them:

Issue 1: Loss Stays Constant

```

# Symptom: Loss doesn't decrease
# Epoch 1: 8.5234
# Epoch 2: 8.5198
# Epoch 3: 8.5201

def debug_static_loss(model, optimizer):
    """
    Checklist when loss won't decrease:
    """

    # 1. Check if gradients are flowing
    print("\n🔍 Checking gradients...")
    for name, param in model.named_parameters():
        if param.grad is not None:

```

```

        grad_norm = param.grad.norm().item()
        print(f"    {name}: {grad_norm:.6f}")
        if grad_norm < 1e-7:
            print(f"        Warning: Very small gradient!")

# 2. Check learning rate
print(f"\n Learning rate: {optimizer.param_groups[0]['lr']}")
if optimizer.param_groups[0]['lr'] < 1e-6:
    print("    Learning rate might be too small!")

# 3. Try increasing learning rate
print("\n Try: Increase learning rate to 1e-3")

# 4. Try more reasoning steps
print(" Try: Increase n_reasoning_steps from 8 to 12")

```

Issue 3: Out of Memory

```

def optimize_memory_usage(model, batch_size):
    """
    Strategies to reduce memory usage.
    """
    print("\n Memory Optimization Strategies:")

    print("\n1. Reduce batch size")
    print(f"    Current: {batch_size}")
    print(f"    Try: {batch_size // 2}")

    print("\n2. Reduce latent length")
    print("    Current: 32")
    print("    Try: 16")

    print("\n3. Enable gradient checkpointing")
    print("""
from torch.utils.checkpoint import checkpoint

def apply_transformer_blocks(self, x):
    for block in self.transformer_blocks:

```

```

        x = checkpoint(block, x)
    return x
"""

print("\n4. Clear cache periodically")
print("""
if batch_idx % 10 == 0:
    torch.cuda.empty_cache()
""")

```

Part 11: Complete End-to-End Example

Let me put it all together with a complete, runnable example:

```

"""
Complete TRM Example: Training and Evaluation
"""

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, Dataset
import math

# [All the classes we defined above would go here:
#  MultiHeadAttention, FeedForward, TransformerBlock, TRM, etc.]

def main():
    """
    Complete training pipeline.
    """
    print("=" * 70)
    print("  TRM: Transformer Reasoning Model")
    print("  Less is More: Recursive Reasoning with Tiny Networks")
    print("=" * 70)

    # Configuration
    vocab_size = 10000

```

```

d_model = 256
n_layers = 4
batch_size = 32
num_epochs = 50
learning_rate = 1e-4
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

print(f"\n Configuration:")
print(f"  Vocabulary size: {vocab_size}")
print(f"  Model dimension: {d_model}")
print(f"  Transformer layers: {n_layers}")
print(f"  Batch size: {batch_size}")
print(f"  Learning rate: {learning_rate}")
print(f"  Device: {device}")

# Create model
print(f"\n🔨 Building model...")
model = create_trm_att(vocab_size, d_model, n_layers)
model = model.to(device)

n_params = model.count_parameters()
print(f"  Parameters: {n_params / 1e6:.2f}M")
print(f"  Model type: TRM-Att (with attention)")

# Optimizer and criterion
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
scheduler = get_lr_scheduler(optimizer, warmup_steps=1000)
criterion = nn.CrossEntropyLoss(ignore_index=0)

# Data (replace with your actual data)
print(f"\n Loading data...")
# train_dataset = YourDataset(...)
# train_loader = DataLoader(train_dataset, batch_size=batch_size, sh
# val_loader = DataLoader(val_dataset, batch_size=batch_size)

# Training
print(f"\n Starting training...")
print("=" * 70)

best_val_accuracy = 0

for epoch in range(num_epochs):
    # Training phase

```

```

model.train()
train_loss = 0
num_batches = 0

# for questions, answers in train_loader:
#     questions = questions.to(device)
#     answers = answers.to(device)
#
#     loss = train_step_with_clipping(
#         model, questions, answers, optimizer, criterion
#     )
#     train_loss += loss
#     num_batches += 1
#     scheduler.step()

# avg_train_loss = train_loss / num_batches

# Evaluation phase
# if (epoch + 1) % 5 == 0:
#     val_accuracy = evaluate_accuracy(model, val_loader, device)
#     current_lr = optimizer.param_groups[0]['lr']
#
#     print(f"\nEpoch {epoch+1}/{num_epochs}")
#     print(f"  Train Loss: {avg_train_loss:.4f}")
#     print(f"  Val Accuracy: {val_accuracy:.2f}%")
#     print(f"  Learning Rate: {current_lr:.6f}")
#
#     if val_accuracy > best_val_accuracy:
#         best_val_accuracy = val_accuracy
#         torch.save(model.state_dict(), 'best_trn_model.pt')
#         print(f"  New best model saved!")

pass # Remove this when uncommenting above

print("\n" + "=" * 70)
print(f" Training complete!")
print(f"   Best validation accuracy: {best_val_accuracy:.2f}%")
print("=" * 70)

if __name__ == "__main__":
    main()

```

The Results That Changed My Mind

When I first read the TRM paper, I was skeptical. “A 7M parameter model beating GPT-4? Come on.”

Then I implemented it and saw the results:

Sudoku Performance

ARC-AGI (Abstract Reasoning)

This isn't just incremental improvement. This is a **paradigm shift**.

The Challenges

Challenge 1: Getting gradients to flow properly through 24 recursive steps.

Solution: Gradient clipping and careful initialization.

Challenge 2: Figuring out the right number of reasoning vs refinement steps.

Too few: underfitting. Too many: no benefit.

Challenge 3: Understanding how to supervise the model without micromanaging z. Answer: Don't! Let it learn.

Conclusion: The Future is Small

We've spent years in an arms race for bigger models. GPT-2 → GPT-3 → GPT-4, each iteration bigger than the last.

TRM shows us a different path: **iterate, don't scale.**

The future of AI isn't just about billion-parameter models trained on internet-scale data. It's about clever architectures that use computation efficiently.

TRM is just the beginning. What else can we achieve by rethinking how models think?

I don't know about you, but I'm excited to find out.

Resources

Less is More: Recursive Reasoning with Tiny Networks

Abstract Hierarchical Reasoning Model (HRM) is a novel approach using two small neural networks recursing at different...

arxiv.org

Thanks for reading! If you build something cool with TRM, I'd love to hear about it



Trm

Tiny Recursive Model

Hrm

Hierarchical Reasoning

Multihead Attention



Follow

Written by azhar

694 followers · 31 following

Data Scientist | Exploring interesting (research paper / concepts). LinkedIn :
<https://www.linkedin.com/in/mohamed-azharudeen/>

Responses (1)



To respond to this story,
get the free Medium app.

Continue in app



Lei Ding

21 hours ago



Very nice explanation!



More from azhar



In azhar labs by azhar

Rotary Positional Embeddings: A Detailed Look and Comprehensive Understanding

Since the “Attention Is All You Need” paper in 2017, the Transformer architecture has been a cornerstone in the realm of Natural Language...

Jan 12, 2024 308 9



In azhar labs by azhar

Understanding GPU Architecture: Basics and Key Concepts

Graphics Processing Units (GPUs) have evolved from being specialized hardware for rendering graphics to becoming the backbone of AI...

Mar 10 22



 In azhar labs by azhar

Model Context Protocol (MCP) in Agentic AI: Architecture and Industrial Applications

By integrating external data and tools through a universal interface, the Model Context Protocol (MCP) is emerging as a key enabler for...

Apr 2  55  2





In azhar labs by azhar

Comprehensive Guide to Setting Up Claude Code on Windows Using WSL

Setting up Claude Code, Anthropic's AI-driven coding assistant, on a Windows system involves leveraging the Windows Subsystem for Linux...

Mar 27  64  2



See all from azhar

Recommended from Medium

 john bessant

Sewing up the competition

The larger-than-life story of Isaac Merritt Singer

2d ago  41  2



 In AI Advances by Nikhil Anand

If you still think AI is magic, read this.

AI might seem mysterious and weird, but it's a lot simpler than you think.

 3d ago  359  13



 In Level Up Coding by Dr. Ashish Bamania 

Tiny Recursive Models Solve ARC-AGI Better Than Most Reasoning LLMs Available Today

Deep dive into the architecture and inner workings of the Tiny Recursive Model (TRM), and explore the reasons why it beats the most...

 4d ago  230  1



 In Artificial Intelligence in Plain English by DrSwarnenduAI

Attention Is Just Kernel Smoothing: The 1956 Statistical Method Behind Transformers

Why the “Revolutionary” Transformer Architecture Is Actually Doing 70-Year-Old Statistics

★ 5d ago 🖱️ 211 💬 4



 Zach Hurley

Finding Hidden Patterns in Data with KMeans Clustering

In the world of data science, not all learning requires labels. Some of the most interesting insights come from simply letting the data...

3d ago



 Civil Learning

The Fastest Web Agent in the World: Browser Use 0.8.0 Changes the Game

I've been testing Browser Use 0.8.0 recently, and wow—this thing is fast. Really quick.

★ Oct 10 🖱️ 99 💬 2



See more recommendations